

ARCHITECTURE

Architecture -- Challenges

Purpose

Generic, reusable Go module for defining, registering, executing, and reporting on challenges (structured test scenarios). Features a plugin-based architecture with 16 built-in assertion evaluators, multi-format reporting (Markdown/JSON/HTML), live WebSocket monitoring, progress-based liveness detection, and a multi-platform user flow automation framework.

Structure

```
pkg/  
  challenge/      Core types: Challenge interface, Config, Result,  
BaseChallenge, ProgressReporter  
  registry/      Challenge registration, dependency ordering  
(Kahn's topological sort)  
  runner/        Execution engine (sequential, parallel,  
pipeline), liveness monitoring  
  assertion/     Assertion engine with 16 built-in evaluators +  
custom evaluator support  
  report/        Report generation: Markdown, JSON, HTML  
  logging/       Structured logging: JSON, Console, Multi,  
Redacting  
  env/           Environment variable handling with redaction  
  httpclient/    Generic REST API client with JWT auth and  
functional options  
  bank/          Challenge bank (load definitions from JSON/YAML)  
  monitor/       Live monitoring with WebSocket dashboard  
  metrics/       Prometheus-compatible challenge metrics  
  plugin/        Plugin system for custom challenge types and  
assertions  
  infra/         Infrastructure bridge to digital.vasic.containers  
module  
  userflow/      Multi-platform user flow automation: 8 adapter  
interfaces, 21 implementations, 19 challenge templates, 12
```

evaluators

cmd/

userflow-runner/ CLI runner for user flow challenges

Key Components

- **challenge.Challenge** -- Interface: ID, Configure, Validate, Execute, Cleanup
- **challenge.BaseChallenge** -- Template method base with Progress-Reporter for liveness detection
- **registry.Registry** -- Challenge registration with dependency ordering via topological sort
- **runner.Runner** -- Execution engine with configurable timeout, stale threshold, and progress monitoring
- **assertion.Engine** -- 16 built-in evaluators (not_empty, contains, min_count, max_latency, etc.)
- **userflow.*Adapter** -- Platform adapters: Playwright, ADB, Tauri, HTTP, gRPC, WebSocket, Gradle, Cargo, npm
- **userflow.*Challenge** -- 19 challenge templates including recorded variants with video verification

Data Flow

```
Registry.Register(challenges) -> Runner.RunAll(ctx, config)
|
|   topological sort by dependencies
|
|   for each challenge:
|       challenge.Configure() -> challenge.Validate() ->
challenge.Execute(ctx)
|
|   ProgressReporter -> Liveness Monitor           Result +
Assertions
|
|   (kill if stale > threshold)
|

assertion.Engine.Evaluate()
|

report.Reporter.Generate()
```

Dependencies

- [digital.vasic.containers](#) -- Container orchestration bridge for test infrastructure
- [github.com/stretchcr/testify](#) -- Test assertions
- [github.com/gorilla/websocket](#) -- WebSocket for live monitoring and WebSocket flow testing

Testing Strategy

209+ tests across pkg/userflow alone. Table-driven tests with testify and race detection. Tests cover challenge lifecycle, dependency ordering, assertion evaluation, report generation, adapter availability detection, and challenge template execution with mock adapters.